Compilatori

UniVR - Dipartimento di Informatica

Fabio Irimie

2° Semestre 2025/2026

Indice

0.1 Storia dei compilatori 2

1 Compilatori **2**

1.1 Modelli ibridi 2

1.2 Fasi della compilazione 3

1.2.1 Lexical analysis 4

1.2.2 Parsing 4

1.2.3 Intermediate code generator 5

1.2.4 Optimization 5

1.2.5 Code generation 5

1.2.6 Front end e back end 6

0.1 Storia dei compilatori

All'inizio i programmi venivano scritti traducendo formule matematiche in codice macchina. Nel 1948 Nathaniel Rochester sviluppò il primo **assembler**, un programma che traduceva istruzioni mnemoniche in codice macchina.

Esempio 0.1. $1 + 2 = 3$ in assembly:

```
MOV R0, 1
MOV R1, 2
ADD R0, R1, R2
MOV R0, 64
STO R2, R0
```

Nel 1953 John Backus sviluppò il primo linguaggio interpretato, chiamato **Speedcoding** che esevuiva dalle 10 alle 20 volte più lentamente dell'assembly scritto a mano, ma era più facile da scrivere e leggere. Successivamente nel 1954-1957 sviluppò il primo **compilatore** per il linguaggio **Fortran** (FORmula TRANslation) che aveva performance paragonabili all'assembly scritto a mano. Questo linguaggio ha avuto un enorme impatto sulla programmazione:

- **Open programming**: La possibilità di programmare viene estesa da un numero limitato di matematici e scienziati a un pubblico più ampio di persone.
- **Portability**: Programmi di alto livello potevano essere ricompilati per un nuovo computer con modifiche minime. Questo ha rimosso la necessità del programmatore di preoccuparsi su che architettura il programma sarebbe stato eseguito.

1 Compilatori

Un compilatore è un programma che legge un altro programma scritto in un linguaggio di programmazione e lo traduce in un altro linguaggio equivalente chiamato **target**. Il compilatore traduce l'intero programma in un altro (solitamente linguaggio macchina) prima dell'esecuzione.

1. Prende l'intero programma in input
2. Lo traduce in un altro linguaggio (solitamente linguaggio macchina)
3. Genera un file eseguibile che prende input e produce output

Il compilatore può anche applicare un processo di ottimizzazione **globale** sul programma, migliorando le prestazioni del codice generato.

1.1 Modelli ibridi

Esistono anche modelli ibridi di compilatori che combinano compilazione e interpretazione:

- **Compilazione in bytecode** che viene poi interpretato da una macchina virtuale. Ad esempio Java con JVM.

  - **Bytecode**: Codice intermedio
  - **Macchina virtuale**: Software che esegue il bytecode

- **Compilazione Just-In-Time (JIT)**: Il codice viene compilato **durante l'esecuzione**, consentendo ottimizzazioni dinamiche basate sull'uso effettivo del programma.

1.2 Fasi della compilazione

Le fasi della compilazione preservano la struttura che Fortran ha introdotto. Ci sono due parti principali nel processo di compilazione:

- **Analisi**: Determina le operazioni implicate dal programma sorgente che sono rappresentate in una struttura ad albero (rappresentazione intermedia). L'informazione sul programma sorgente è salvata nella **symbol table**, che contiene informazioni su variabili, funzioni, tipi, ecc e permette di comunicare tra le fasi di compilazione. Questa fase è composta da:
  - **Lexical analysis**: Suddivide il codice sorgente (una lista di caratteri) in token (una lista di parole chiave, identificatori, operatori, ecc.)
  - **Parsing**: Analizza la struttura sintattica del programma, costruendo un albero sintattico astratto (AST) che rappresenta la struttura gerarchica del programma.
  - **Semantic analysis**: Verifica la correttezza semantica del programma, controllando tipi, dichiarazioni, e altre regole del linguaggio.
- **Sintesi**: Prende la rappresentazione intermedia ad albero e traduce tutte le operazioni nel programma target. Questa fase è composta da:
  - **Optimization**: Migliora le prestazioni del codice generato, ad esempio eliminando codice morto o riducendo il numero di istruzioni.
  - **Code generation**: Traduce l'AST ottimizzato in codice macchina o in un altro linguaggio target **senza cambiare la semantica del programma originale**.

Per generare codice eseguibile c'è bisogno di altri programmi in cui il compilatore è solo una parte del processo:

- **Preprocessor**: Prende in input il codice sorgente e lo trasforma, ad esempio espandendo macro o includendo file header.
- **Compilatore**: Traduce il codice sorgente preprocessato in codice oggetto (codice macchina non ancora collegato).
- **Assembler**: Traduce il codice oggetto in codice macchina eseguibile.
- **Linker**: Combina più file oggetto in un unico file eseguibile, risolvendo le dipendenze tra i vari moduli.

1.2.1 Lexical analysis

Il processo di lexical analysis, o tokenizzazione, è il primo passo nella fase di analisi e consiste nel riconoscere le "parole" del programma:

Esempio 1.1. Bisogna riconoscere:

```
This is a Compiler book.
```

Bisogna riconoscere anche che corrispondano alle regole del linguaggio, quindi la seguente stringa non è valida:

```
thi sis aCompil erb ook.
```

Il lexical analyzer legge lo stream di caratteri e li raggruppa in sequenze significative chiamate **lexemes** e per ogni lexeme produce in output un **token** della forma:

```
<token_name, token_value>
```

che viene messo nella tabella dei simboli.

Esempio 1.2. Consideriamo la seguente riga di codice:

```
position = initial + rate * 60
```

Il lexical analyzer raggruppa i caratteri nei seguenti lexemes e tokens:

Lexeme	Token
position	<id, 1>
=	<=>
initial	<id, 3>
+	<+>
rate	<id, 3>
*	<*>
60	<60>

Il `token_value` è un riferimento alla tabella dei simboli:

id	name
1	position
2	initial
3	rate

La sequenza di token ritornata dal lexical analyzer è:

```
<id, 1> <=> <id, 2> <+> <id, 3> <*> <60>
```

1.2.2 Parsing

Il parsing è una fase del lexical analysis che si occupa di analizzare la struttura sintattica del programma, costruendo un albero sintattico astratto (AST) che rappresenta la struttura gerarchica.

Esempio 1.3. La sequenza di token dell'esempio precedente viene rappresentata nel seguente albero sintattico:

```
<id, 1> <=> <id, 2> <+> <id, 3> <*> <60>
```

$$\Downarrow$$

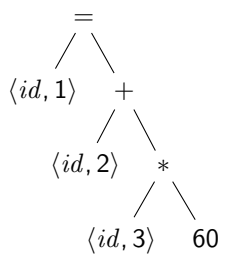

Figura 1: Syntax Tree

Questa fase controlla gli **errori semantici** (Type Checking) e ottiene le informazioni dei tipi per le fasi successive. Il type checking controlla i tipi degli operandi (potrebbe anche imporre delle conversioni di tipo) e controlla che le operazioni siano valide per i tipi degli operandi. Ad esempio controlla che non ci siano numeri reali per indicizzare un array.

1.2.3 Intermediate code generator

Il **codice intermedio** è generato come un programma per una **macchina astratta**. Questo codice dovrebbe essere facile da generare e tradurre nel programma target.

1.2.4 Optimization

L'ottimizzazione **indipendentemente dalla macchina** cerca di migliorare il codice intermedio in modo da ottenere codice macchina migliore, ad esempio più veloce o che utilizza meno memoria o che consuma meno energia o rete, ecc... . Compilatori diversi possono applicare ottimizzazioni diverse.

1.2.5 Code generation

Questa fase genera il codice target che consiste in codice assembly.

- Per ogni variabile e risultato intermedio, viene assegnato un **registro** o una posizione in memoria. L'**allocazione della memoria** viene fatta in questa fase, oppure nella fase di intermediate code generation.
- Le istruzioni sono tradotte in sequenze di istruzioni assembly.

Esempio 1.4. Il seguente codice verrà poi tradotto in assembly:

```
t1 = id3 * 60
id1 = id2 + t1
```

$\Downarrow$

```
LDF R2, id3
MULF R2, R2, 60
LDF R1, id2
ADDF R1, R1, R2
STOF R1, id1
```

1.2.6 Front end e back end

Il **front end** del compilatore si occupa dell'analisi, mentre il **back end** si occupa della sintesi. Il front end è indipendente dalla macchina, mentre il back end è specifico per la macchina target. Questo permette di riutilizzare il front end per diversi target, ad esempio un compilatore che supporta sia x86 che ARM può avere lo stesso front end e due back end diversi.